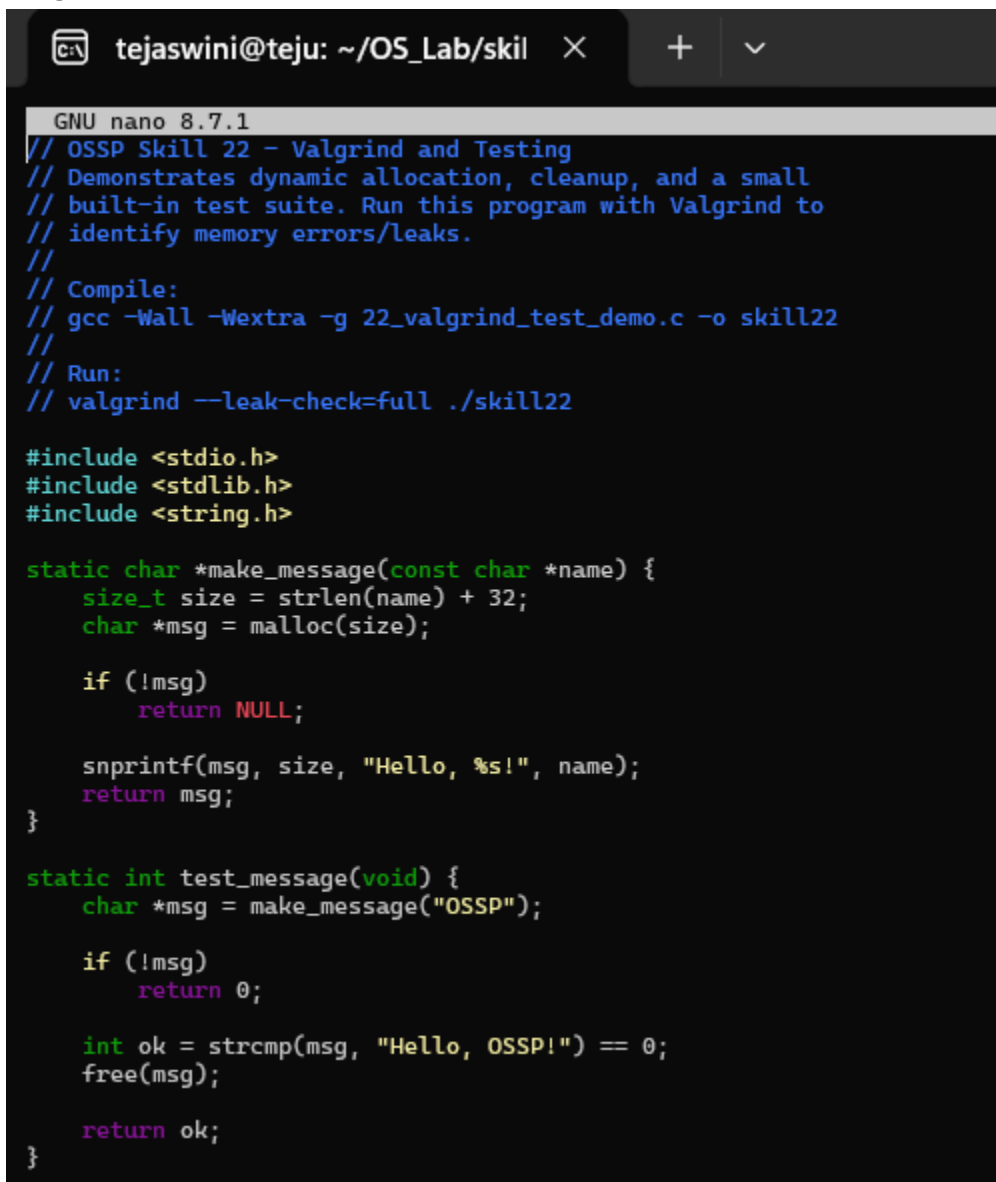


Skill-22

Rollno:2520030011

Program:



The image shows a terminal window with a dark background. At the top, a tab is labeled 'tejaswini@teju: ~/OS_Lab/skil'. Below the tab, the terminal shows the GNU nano 8.7.1 editor. The code is a C program for Skill 22, demonstrating dynamic allocation, cleanup, and a small built-in test suite. It includes headers for stdio.h, stdlib.h, and string.h. The program defines a static function 'make_message' that takes a name and returns a dynamically allocated string. It also defines a static function 'test_message' that tests the 'make_message' function. The main function is not visible in the screenshot.

```
GNU nano 8.7.1
// OSSP Skill 22 - Valgrind and Testing
// Demonstrates dynamic allocation, cleanup, and a small
// built-in test suite. Run this program with Valgrind to
// identify memory errors/leaks.
//
// Compile:
// gcc -Wall -Wextra -g 22_valgrind_test_demo.c -o skill22
//
// Run:
// valgrind --leak-check=full ./skill22

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

static char *make_message(const char *name) {
    size_t size = strlen(name) + 32;
    char *msg = malloc(size);

    if (!msg)
        return NULL;

    snprintf(msg, size, "Hello, %s!", name);
    return msg;
}

static int test_message(void) {
    char *msg = make_message("OSSP");

    if (!msg)
        return 0;

    int ok = strcmp(msg, "Hello, OSSP!") == 0;
    free(msg);

    return ok;
}
```

```

static int test_multiple_allocations(void) {
    char *a = make_message("Linux");
    char *b = make_message("Ubuntu");

    if (!a || !b) {
        free(a);
        free(b);
        return 0;
    }

    int ok = strstr(a, "Linux") && strstr(b, "Ubuntu");

    free(a);
    free(b);

    return ok;
}

int main(void) {
    printf("=== Skill 22: Valgrind + Testing ===\n");

    int passed = 0;
    int total = 2;

    if (test_message()) {
        printf("[PASS] test_message\n");
        passed++;
    } else {
        printf("[FAIL] test_message\n");
    }

    if (test_multiple_allocations()) {
        printf("[PASS] test_multiple_allocations\n");
        passed++;
    } else {
        printf("[FAIL] test_multiple_allocations\n");
    }

    printf("\nResult: %d/%d tests passed.\n", passed, total);
    printf("Run with Valgrind to verify memory cleanup.\n");

    return passed == total ? 0 : 1;
}
|

```

Output:

```

tejaswini@teju:~/OS_Lab/skill$ nano skill_22.c.c
tejaswini@teju:~/OS_Lab/skill$ gcc skill_22.c.c -o skill_22
tejaswini@teju:~/OS_Lab/skill$ ./skill_22
=== Skill 22: Valgrind + Testing ===
[PASS] test_message
[PASS] test_multiple_allocations

Result: 2/2 tests passed.
Run with Valgrind to verify memory cleanup.
tejaswini@teju:~/OS_Lab/skill$ |

```